Техническое задание

на разработку платформы для изучения и практики курса по криптографии в МИФИ

Версия: 0.5

Дата: 24.10.2026

Роль	ФИО	Подпись / Дата
Разработал:	Кокорев В.Д.	
Согласовано:		
Утверждено:		

г. Москва, 2026

Лист регистрации изменений [TS.CHANGES]

Версия	Дата	Автор	Изменённый элемент	Х-р изменения	Описание изменения	Утвердил
0.5	24.10.2026	Кокорев В.Д.	Весь документ	Добавлено	Первичный выпуск (черновая версия Спринта 1). Детализация требований — Спринт 2; v1.0 — до 30.10.2026.	

Содержание

Лист регистрации изменений 1

1 Введение [TS.INTRO] 3
- 1.1 Наименование [TS.INTRO.NAME] . . . 3
- 1.2 Цели и назначение разработки [TS.INTRO.PURPOSE] . . . 3
- 1.3 Область применения [TS.INTRO.SCOPE] . . . 3
- 1.4 Термины и определения [TS.INTRO.TERMS] . . . 4

2 Общее описание системы [TS.DESC] 6
- 2.1 Концепция [TS.DESC.CONCEPT] . . . 6
- 2.2 Архитектура и взаимодействие [TS.DESC.ARCH] . . . 6
- 2.3 Известные ограничения реализации [TS.DESC.LIMIT] . . . 8

3 Требования к системе [TS.REQ] 9
- 3.1 Функциональные требования [TS.REQ.FUNC] . . . 9
- 3.2 Требования к интерфейсам [TS.REQ.INT] . . . 11
- 3.3 Требования к надёжности [TS.REQ.REL] . . . 11
- 3.4 Требования к безопасности [TS.REQ.SEC] . . . 12
- 3.5 Требования к производительности [TS.REQ.PERF] . . . 12

4 Требования к документации и разработке [TS.DOC] 12

5 Порядок контроля и приёмки [TS.TEST] 13
- 5.1 Мероприятия по верификации [TS.TEST.VER] . . . 13
- 5.2 Критерии приёмки [TS.TEST.ACCEPT] . . . 14

1 Введение [TS.INTRO]

1.1 Наименование [TS.INTRO.NAME]

Полное наименование:

«Платформа для изучения и практики курса по криптографии в МИФИ».

Краткое наименование:

криптотренажёр mephi-12.

Далее по тексту используется наименование «Система» (определение — в п. 1.4).

1.2 Цели и назначение разработки [TS.INTRO.PURPOSE]

Система предназначена для практической подготовки студентов НИЯУ МИФИ по криптографическим методам защиты информации — ранцевым криптосистемам (на супервозрастающей последовательности и на последовательности степеней), бэкдор-атаке на ранцевую криптосистему и схеме Эль-Гамаля. Подготовка осуществляется через решение автоматически генерируемых и автоматически проверяемых на сервере задач, объединяемых в сессии задач с ограничением времени, в контексте учебных групп и именованных конфигураций наборов задач.

Цели разработки.

- Автоматическая генерация учебных криптографических задач и автоматическая проверка решений на сервере, без участия преподавателя в рутинной проверке.
- Организация практики в форме сессий задач с лимитом времени на задачу и автоматическим провалом просроченных задач.
- Учёт результатов решения по учебным группам средствами конфигураций наборов задач и функции модерации (просмотр сессий студентов).
- Разграничение возможностей пользователей по ролям (USER, ADMIN).
- Демонстрация на практике уязвимостей криптосистем — встроенного бэкдора в ранцевой криптосистеме — как элемента компетенций по криптоанализу.

1.3 Область применения [TS.INTRO.SCOPE]

Система применяется в учебном процессе кафедры НИЯУ МИФИ: поддержка лабораторных работ и практикума по дисциплинам, включающим криптографические методы защиты информации. Система эксплуатируется как серверное приложение (backend) учебного назначения; взаимодействие с ней осуществляется по REST API (порт 8083, базовый путь /api); клиентская часть в объём настоящего технического задания не входит.

1.4 Термины и определения [TS.INTRO.TERMS]

Применяемые в настоящем техническом задании термины соответствуют сквозному глоссарию комплекта документации (методика идентификаторов v1.0); ниже приведены ключевые термины с теми же формулировками.

Система:

«Платформа для изучения и практики курса по криптографии в МИФИ» (учебный криптотренажёр, репозиторий mephi-12/backend): монолит Kotlin 1.9.25 + Spring Boot 3.4.0 (Web, JPA, Security, Validation) с БД PostgreSQL 17, развёртываемый средствами Docker/docker-compose; REST API — базовый путь /api, порт 8083.

Пользователь:

Учётная запись (сущность User): уникальный email, имя, BCrypt-хеш пароля; опционально привязан к одной группе (User.group); идентифицируется в запросах по клейму userId своего JWT.

Студент:

Пользователь в роли USER, решающий сессии задач; все операции создания сессий и отправки решений выполняются от имени студента (владельца активного JWT).

Роль:

Значение authority в JWT: ROLE_USER (выдаётся каждому при регистрации/логине) и ADMIN. SecurityConfig: пути /api/auth/** и /api/public/** открыты, все остальные требуют роль USER.

JWT:

Access-токен стандарта JSON Web Token (библиотека jjwt), выпускается при регистрации и логине (JwtHelper); клеймы: userId, authorities, permissions; срок жизни задаётся переменной окружения ACCESS_TOKEN_LIFETIME (в днях); проверяется фильтром JwtFilter на каждом запросе.

Сессия задач:

Период решения студентом набора задач (сущность ProblemSession): создаётся POST /api/tasks/session?configuration=<имя> по включённой для группы студента конфигурации; у студента не более одной активной сессии (sessionState = NEW, повторное создание отклоняется); время жизни задачи — 20 минут, по истечении задача автоматически проваливается планировщиком автопровала; сессия закрывается, когда не осталось задач в состоянии NEW.

Задача:

Элемент сессии (сущность Problem): JSON-условие (statement), тип (ProblemType) и состояние (ProblemState: NEW «Задание решается», SOLVED «Задание верно решено», FAILED «Задание решено неверно»); проверка решения — PUT /api/tasks/

submit?statementId=<UUID>; владелец и активность сессии проверяются при отправке.

Тип задачи:

Значение перечисления ProblemType: BACKPACK_CODE_SUPER_INCREASING (ранцевая КС на супервозрастающей последовательности), BACKPACK_CODE_DEGREES (ранцевая КС на степенях), BACKPACK (ранцевая КС в составе сессии), BACKPACK_BACKDOOR (бэкдор в ранцевой КС), EL_GAMAL (КС Эль-Гамаля). В конфигурациях сессий задачи задаются строковыми квалификаторами «Backpack» и «ElGamal»; типы BACKPACK_CODE_* используются редакционными (editorial) генерациями GET/POST /api/tasks/backpack/editorial.

Ранцевая криптосистема (КС):

Криптосистема Меркля–Хеллмана на задаче о рюкзаке: лёгкий рюкзак L (супервозрастающая последовательность), модуль $n = \Sigma L + 1$, множитель ω, взаимно простой с n; публичный тяжёлый рюкзак $H = (L \cdot \omega) \bmod n$; шифрование — сумма элементов H по единичным битам сообщения; расшифрование — умножением на $\omega^{-1} \bmod n$.

Супервозрастающая последовательность:

Последовательность, каждый элемент которой строго больше суммы всех предыдущих; в типе BACKPACK_CODE_SUPER_INCREASING из неё строится лёгкий рюкзак.

Бэкдор:

Секрет, встроенный в публичный ключ ранцевой КС на степенях (эндпоинт GET /api/backdoor/editorial): десятичные записи ω и n «зашиты» в младшие разряды элементов тяжёлого рюкзака (цифра $H_i \bmod 10$), что позволяет владельцу секрета восстановить закрытый ключ из открытого.

Схема Эль-Гамаля:

Криптосистема на мультипликативной группе Z_p^*: ключи (p, g, закрытый x, открытый $y = g^x \bmod p$), шифрование с сеансовым показателем k, расшифрование через x; учебные генерации — GET /api/tasks/eigamal?bitLength=<N> (задача) и GET /api/tasks/eigamal/editorial?bitLength=<N> (разбор).

Конфигурация задач:

Сущность ProblemConfiguration: имя + упорядоченный список квалификаторов типов задач (taskServices), задающий состав и порядок задач в создаваемой сессии; доступность группе — через UserGroupProblemConfiguration.enabled (включённые конфигурации группы: GET /api/tasks/groups/configurations?groupId=<UUID>).

Группа пользователей:

Сущность UserGroup (учебная группа): объединяет пользователей (User.group) и свя-

зывается с конфигурациями задач; список групп — GET /api/users/groups, привязка себя к группе — PUT /api/users/me/groups?group=<имя>.

Планировщик автопровала:

Фоновый процесс ProblemAutoFailScheduler (раз в 60 с): переводит задачи в состоянии NEW старше 20 минут (по времени создания) в FAILED и закрывает сессии, в которых не осталось задач в состоянии NEW.

2 Общее описание системы [TS.DESC]

2.1 Концепция [TS.DESC.CONCEPT]

Система реализуется как монолитное серверное приложение со слоистой архитектурой: REST-сервис на Kotlin 1.9.25 / Spring Boot 3.4.0 (стартеры Web, JPA, Security, Validation) с СУБД PostgreSQL 17; развёртывание — средствами Docker/docker-compose (контейнер БД с контролем готовности, затем контейнер приложения). Система предоставляет 14 действующих эндпоинтов в 7 контроллерах.

Студенты регистрируются и вступают в учебные группы. С каждой группой связываются именованные конфигурации задач — упорядоченные наборы типов задач; студент запускает сессию задач по включённой конфигурации своей группы и получает автоматически сгенерированные условия. Решения отправляются на сервер и проверяются автоматически, с вынесением вердикта по каждой задаче (SOLVED — верно решено, FAILED — решено неверно). На решение каждой задачи отводится 20 минут; по истечении лимита задача автоматически проваливается фоновым планировщиком, что исключает неограниченное время решения.

Роли: USER — все операции студента (решение задач); ADMIN — административные функции, MODERATOR как отдельная роль не выделяется (функция модерации — просмотр сессий всех студентов); об ограничениях текущей реализации доступа — п. 2.3.

2.2 Архитектура и взаимодействие [TS.DESC.ARCH]

Основной сценарий использования.

1. Регистрация: студент передаёт email, имя и пароль; сервер валидирует поля (email 4–120 символов, пароль 8–255) и при успехе сразу выдаёт JWT.

2. Логин: при повторном входе студент аутентифицируется парой «email + пароль» и получает JWT.

3. Аутентифицированный доступ: все дальнейшие запросы передаются с заголовком Authorization: Bearer <JWT>; клеймы userId и authorities идентифицируют студента и его роль.

4. Выбор группы: студент просматривает список учебных групп и вступает в одну из них по названию; без группы сессия не создаётся.

5. Просмотр конфигураций: студент просматривает конфигурации задач своей группы (имя набора, состав типов, признак включения).

6. Запуск сессии: студент создаёт сессию по имени конфигурации; сервер генерирует задачи в порядке, заданном конфигурацией (квалификаторы «Backpack» и «ElGamal»), и возвращает сессию в состоянии NEW со списком задач.

7. Получение задачи: условие каждой задачи (JSON), тип и состояние NEW входят в состав сессии; тип задачи определяется конфигурацией (ранцевая КС либо схема Эль-Гамаля).

8. Решение: студент готовит решение — восстанавливает или вычисляет параметры криптосистемы по условию.

9. Отправка решения: решение отправляется по идентификатору задачи (statementId); сервер проверяет владельца и активность сессии и передаёт решение проверщику соответствующего типа.

10. Вердикт: задача переводится в SOLVED либо в FAILED (с диагностическим сообщением о неверном параметре); когда задач в состоянии NEW не остаётся, сессия завершается (исчерпание задач либо автопровал по таймеру — см. фоновые сценарии).

Типы задач (что решает студент).

BACKPACK_CODE_SUPER_INCREASING (ранцевая КС на супервозрастающей последовательности):
студент получает условие с частично скрытым лёгким ранцем (часть элементов скрыта, часть параметров отсутствует) и восстанавливает все параметры криптосистемы: лёгкий ранец, модуль, множитель ω, тяжёлый ранец, обратный множитель ω^{-1}, шифртекст и расшифрованное сообщение (расшифрование — жадным алгоритмом по супервозрастающей последовательности).

BACKPACK_CODE_DEGREES (ранцевая КС на последовательности степеней):
то же, но лёгкий ранец строится из элементов вида p^i (степени основания p); студент выполняет шифрование и расшифрование в этой конструкции, восстанавливая тот же набор параметров.

BACKPACK_BACKDOOR (бэкдор в ранцевой КС, учебная форма атаки Шамира):
студент получает публичный тяжёлый ранец, в младшие разряды элементов которого «зашиты» десятичные записи секретных ω и n, и убеждается, что владелец секрета восстанавливает закрытый ключ из открытого. Тип используется только редакционной демонстрацией и в сессиях не участвует.

EL_GAMAL (схема Эль-Гамаля):

студент получает открытые параметры и сообщение (p, g, x, m), вычисляет открытый ключ $y = g^x \bmod p$, выбирает сеансовый показатель k, строит компоненты шифртекста c_1 и c_2 и выполняет расшифрование; проверка решения — по шагам, с сообщением о первом неверном промежуточном значении.

Фоновые сценарии. Планировщик автопровала выполняется каждые 60 с: задачи в состоянии NEW, созданные более 20 минут назад, переводятся в FAILED (момент истечения определяется по времени создания задачи); сессии, в которых не осталось задач в состоянии NEW, закрываются. Автопровал не зависит от действий студента, в том числе от его отключения от Системы.

Учебные (editorial) сценарии. Редакционные эндпоинты предназначены для самоподготовки и подготовки методических материалов: эталонная ранцевая задача выбранного типа с полным решением и возможностью задания параметров (GET/POST /api/tasks/backpack/editorial), демонстрационный пример со встроенным бэкдором (GET /api/backdoor/editorial), пошаговый разбор схемы Эль-Гамаля (GET /api/tasks/eigamal/editorial) и задание Эль-Гамаля для самостоятельного решения (GET /api/tasks/eigamal). Результаты редакционных генераций формируются на лету и не записываются в базу данных.

2.3 Известные ограничения реализации [TS.DESC.LIMIT]

Ниже честно фиксируются ограничения подтверждённой вычиткой кода реализации; они являются материалом для формальных инспекций и ревизии требований Спринта 2 (детализация — в CT DOC-05).

1. Роль ADMIN не назначается: JWT всегда выпускается без административной authority, т.е. получить роль ADMIN через API невозможно; эндпоинт модерации GET /api/moderator/tasks/session в текущей реализации доступен любому аутентифицированному пользователю с ролью USER.

2. История решений: GET /api/users/me/tasks обслуживается устаревшим сервисом (помечен как deprecated) и возвращает все сохранённые ранцевые задачи, а не только задачи текущего студента.

3. Управление группами и конфигурациями: API покрывает только чтение конфигураций и вступление в группу; создание, изменение и удаление групп и конфигураций, а также переключение признака включения выполняются на уровне базы данных.

4. Жёстко зафиксированные параметры: лимит времени на задачу (20 минут) зашит в SQL-запросе планировщика и не вынесен в конфигурацию; параметры генерации задач Эль-Гамаля в сессии зафиксированы (bitLength = 5) и не настраиваются конфигурацией.

3 Требования к системе [TS.REQ]

В настоящей редакции (v0.5) требования приводятся тезисно, на уровне намерений разработки, и соответствуют фактическим возможностям реализованной Системы. Полный набор требований с атомарной декомпозицией и присвоением идентификаторов SR-XXX формируется в Спринте 2 в системных требованиях (DOC-05).

3.1 Функциональные требования [TS.REQ.FUNC]

[TS.REQ.FUNC.AUTH.REG]. Система должна обеспечивать регистрацию нового пользователя по адресу электронной почты, имени и паролю с серверной валидацией полей (длина email 4–120 символов, пароля 8–255 символов) и отклонением повторной регистрации по занятому адресу. Успешная регистрация должна завершаться немедленной выдачей JWT.

[TS.REQ.FUNC.AUTH.LOGIN]. Система должна обеспечивать аутентификацию (логин) по паре «email + пароль» со сверкой пароля по хешу и выдачей JWT; при неверном пароле — отказ с кодом 401.

[TS.REQ.FUNC.AUTH.JWT]. Система должна требовать предъявления действительного JWT (заголовок Authorization: Bearer) для всех операций, кроме регистрации и логина; клеймы userId и authorities должны идентифицировать текущего студента (профиль пользователя) и его роль во всех доменах Системы.

[TS.REQ.FUNC.GROUP.LIST]. Система должна предоставлять перечень всех учебных групп (идентификатор, название, состав участников).

[TS.REQ.FUNC.GROUP.JOIN]. Система должна позволять студенту вступить в учебную группу по её названию (привязка — к одной группе) и должна отклонять создание сессии, если студент не состоит в группе.

[TS.REQ.FUNC.CONFIG.VIEW]. Система должна предоставлять по группе перечень конфигураций задач (имя, состав типов, признак включения); создание сессии должно допускаться только по включённой конфигурации группы студента.

[TS.REQ.FUNC.SESSION.CREATE]. Система должна создавать сессию задач по имени конфигурации: генерировать условия задач в порядке, заданном конфигурацией, и возвращать студенту сессию в состоянии NEW с JSON-условиями, типами и состояниями задач. При наличии у студента активной (незавершённой) сессии создание новой должно отклоняться.

[TS.REQ.FUNC.SESSION.TIME]. Система должна ограничивать время решения каждой задачи 20 минутами и по истечении лимита автоматически переводить задачу в FAILED фоновым планировщиком с периодом срабатывания 60 с, независимо от действий студента.

[TS.REQ.FUNC.SESSION.SUBMIT]. Система должна принимать решение задачи по её идентификатору (statementId) при условии, что задача принадлежит текущему студенту и его сессия активна, передавать решение проверщику соответствующего типа и выносить вердикт SOLVED либо FAILED с диагностическим сообщением при ошибке. Сессия должна закрываться, когда в ней не остаётся задач в состоянии NEW.

[TS.REQ.FUNC.PROB.BACKPACK.SI]. Для задач «ранцевая КС на супервозрастающей последовательности» Система должна генерировать условие с частично скрытыми параметрами и проверять решение по всем восстанавливаемым величинам: элементам лёгкого ранца (строго сверхвозрастающим), модулю, взаимной простоте множителя ω с модулем, тяжёлому ранцу, обратному множителю, шифртексту и расшифрованному сообщению.

[TS.REQ.FUNC.PROB.BACKPACK.DEG]. Для задач «ранцевая КС на степенях» Система должна строить лёгкий ранец из степеней основания p и выполнять генерацию условия и проверку решения по той же схеме параметров ранцевой КС.

[TS.REQ.FUNC.PROB.BACKDOOR]. Система должна генерировать демонстрационный пример ранцевой КС со встроенным бэкдором: десятичные записи секретных ω и модуля размещаются в младших разрядах элементов тяжёлого ранца; при невозможности построить пример за отведённое число попыток — возвращать диагностическую ошибку. Тип используется только редакционной генерацией, в сессиях не участвует.

[TS.REQ.FUNC.PROB.ELGAMAL]. Для задач «схема Эль-Гамаля» Система должна генерировать учебное задание (p, g, x, m) и проверять решение по шагам: открытый ключ y, сеансовый показатель k, компоненты шифртекста c_1 и c_2, расшифрованное сообщение; при ошибке — возвращать сообщение о первом неверном промежуточном значении.

[TS.REQ.FUNC.EDITORIAL]. Система должна предоставлять редакционные (editorial) генерации для самоподготовки: эталонную ранцевую задачу выбранного типа с полным решением (с возможностью задания параметров), демонстрационный пример с бэкдором, пошаговый разбор схемы Эль-Гамаля и задание Эль-Гамаля для самостоятельного решения. Результаты редакционных генераций не должны записываться в базу данных.

[TS.REQ.FUNC.MOD.VIEW]. Система должна предоставлять функцию модерации — просмотр сессий задач всех студентов (состояния сессий и их задач) для контроля учебного прогресса.

[TS.REQ.FUNC.HISTORY]. Система должна предоставлять студенту просмотр истории решённых задач (об известном ограничении текущей реализации — п. 2.3).

3.2 Требования к интерфейсам [TS.REQ.INT]

[TS.REQ.INT.REST]. Все внешние интерфейсы Системы должны следовать единой конвенции REST: базовый путь /api (порт 8083), формат обмена данными — JSON, идентификация ресурсов — по UUID.

[TS.REQ.INT.BEARER]. Аутентификация запросов должна выполняться заголовком Authorization: Bearer <JWT>; без аутентификации должны быть доступны только операции регистрации и логина (/api/auth/**).

[TS.REQ.INT.CODES]. Система должна единообразно применять коды состояния: 401 — отсутствие, истечение или неверность JWT (с требованием авторизоваться заново) и неверный пароль; 403 — недостаточность роли; 400 — невалидное тело запроса (со списком полей); 404 — отсутствие запрашиваемого ресурса. Ошибки должны возвращаться в едином JSON-формате (код, сообщение, время).

3.3 Требования к надёжности [TS.REQ.REL]

[TS.REQ.REL.AUTUFAIL]. Автоматический провал просроченных задач должен обеспечиваться фоновым планировщиком и не должен зависеть от действий студента, в том числе от его отключения от Системы.

[TS.REQ.REL.PERSIST]. Состояние пользователей, групп, конфигураций, сессий и задач должно храниться в PostgreSQL и сохраняться при перезапуске приложения; после перезапуска незавершённые сессии должны продолжать обслуживаться, включая автопровал по таймеру.

[TS.REQ.REL.SCHED]. Истечение времени задачи должно определяться по времени её создания, а не по факту срабатывания планировщика: пропуск или задержка отдельного цикла планировщика не должны приводить к некорректным вердиктам — задача проваливается при первом же цикле после истечения лимита.

[TS.REQ.REL.DEPLOY]. Развёртывание docker-compose должно обеспечивать старт приложения только после готовности СУБД (healthcheck) и сохранность данных базы между перезапусками (том данных).

3.4 Требования к безопасности [TS.REQ.SEC]

[TS.REQ.SEC.JWT]. JWT должен подписываться алгоритмом HMAC-SHA256; срок жизни токена должен задаваться конфигурацией (в днях) через переменную окружения.

[TS.REQ.SEC.PASSWORDS]. Пароли пользователей не должны храниться и передаваться в открытом виде: хранение — только в виде BCrypt-хеша, сверка — по хешу.

[TS.REQ.SEC.ROLES]. Система должна разграничивать возможности по ролям (USER, ADMIN), передаваемым в клеймах authorities токена; фактическое назначение роли ADMIN в текущей реализации — см. п. 2.3, уточнение правил доступа — предмет СТ.

[TS.REQ.SEC.SECRETS]. Секреты и параметры окружения (секрет подписи JWT JWT_SECRET, срок жизни токена ACCESS_TOKEN_LIFETIME, параметры подключения к базе данных) должны передаваться только через переменные окружения и не должны включаться в код или репозиторий.

3.5 Требования к производительности [TS.REQ.PERF]

[TS.REQ.PERF.CRUD]. Операции регистрации, логина, просмотра групп и конфигураций, создания сессии и отправки решения должны выполняться за время, не создающее ощутимых задержек для интерактивной учебной работы.

[TS.REQ.PERF.GEN]. Генерация учебных задач, включая генерации с параметром bitLength и переборное построение примера с бэкдором (ограниченное число попыток), должна укладываться в интерактивно приемлемое для учебного процесса время.

Примечание: количественные показатели производительности (время отклика, пропускная способность) в настоящей редакции не устанавливаются; качественные требования уточняются количественными показателями в СТ (DOC-05, раздел [SR.PERF]).

4 Требования к документации и разработке [TS.DOC]

[TS.DOC.SET]. Разработка должна сопровождаться комплектом документации в стиле ДО-178C/КТ-178C, охватывающим цепочку «требования → реализация → верификация»: DOC-00 (README комплекта), DOC-01 (описание назначения документов), DOC-02 (настоящее ТЗ), DOC-03 (ПУКПО — план управления конфигурацией), DOC-04 (процедура формальной инспекции), DOC-05 (системные требования), DOC-06 и DOC-06.1–06.6 (программные требования — сводно и по подсистемам), DOC-07 (архитектура ПО), DOC-08 (справочник REST API), DOC-09 (матрица трассировки SR → SWR → TC), DOC-10 (план верификации ПО), DOC-11 (план интеграционного тестирования), DOC-13.1–13.6 (протоколы формальных инспекций), DOC-14 (количественный анализ покрытия требований).

[TS.DOC.TRACE]. Требования должны быть трассируемы: каждое системное требование (SR-XXX) — минимум на одно программное требование (SWR-XXX) и минимум на один верификационный кейс (TC-XXX); каждое SWR — минимум на одно SR и один TC. Цепочка SR → SWR → TC ведётся в матрице трассировки DOC-09.

[TS.DOC.TECH]. В качестве технологий реализации фиксируются: язык Kotlin 1.9.25, платформа Spring Boot 3.4.0 (Web, JPA, Security, Validation, Actuator), СУБД PostgreSQL 17, аутентификация JWT (jjwt), развёртывание Docker/docker-compose.

[TS.DOC.NOTEST]. Тестовый код не пишется и прогоны не выполняются (решение от 20.10.2026): верификация обеспечивается формальными инспекциями и описательными планами DOC-10/DOC-11 без исполнения тестовых прогонов.

5 Порядок контроля и приёмки [TS.TEST]

Контроль и приёмка осуществляются на основании комплекса верификационных мероприятий по документации и Системе; исполнение тестовых прогонов не предусматривается (п. 4).

5.1 Мероприятия по верификации [TS.TEST.VER]

Формальные инспекции:

инспекции документов комплекта по процедуре DOC-04 (по Фагану), с ротацией рецензентов по чужим доменам; по документам приоритета оформляются протоколы DOC-13.1–13.6; выявленные несоответствия получают идентификаторы DEF-XXX и закрываются до конца спринта, следующего за протоколом.

Анализ трассировки:

проверка полноты цепочки SR → SWR → TC в матрице DOC-09; нарушение инвариантов трассировки (п. 4, [TS.DOC.TRACE]) является блокирующим замечанием.

Верификационные кейсы:

план верификации DOC-10 определяет описательные кейсы TC-XXX — позитивные и негативные (401 без JWT, 403 без роли, 400 при невалидном теле) — с ожидаемыми результатами по каждому требованию; исполнение прогонов не выполняется.

Сценарии интеграционного тестирования:

план DOC-11 описывает сквозные цепочки (регистрация → группа → сессия → решение → вердикт; автопровал по таймеру; редакционные генерации; модерация) со ссылками на кейсы TC-XXX из DOC-10, без исполнения прогонов.

5.2 Критерии приёмки [TS.TEST.ACCEPT]

Работа считается принятой при выполнении следующих условий:

- все документы комплекта доведены до версии v1.0 и утверждены (листы регистрации изменений заполнены);
- формальные инспекции по всем документам приоритета проведены с результатом ПРИНЯТО или ПРИНЯТО С ЗАМЕЧАНИЯМИ (протоколы DOC-13.1–13.6 оформлены);
- все замечания DEF-XXX закрыты в установленные сроки;
- матрица трассировки DOC-09 полна — инварианты «каждое SR имеет минимум одно SWR и один TC; каждое SWR — минимум одно SR и один TC» выполнены;
- инварианты комплекта, установленные Реестром документов, выполнены; полнота покрытия подтверждена количественным анализом DOC-14.